PRODUCT SERVICE
REST API Documentation

Step 9 — API Documentation

Item	Value
Service	Product Service
API Version	v1
Base URL	http://localhost:8082
Format	JSON
Port	8082
Base Endpoint	/api/v1/products

This document covers the Product Service REST API, all required endpoints, request and response examples, validation, error handling, status codes, Postman testing scenarios, API summary, and versioning rationale.

1. API Overview

The Product Service provides REST endpoints to create, read, update, partially update, and delete products. The API uses JSON and URL-based versioning under **/api/v1**.

Service Details

Item	Value
Service name	Product Service
Base URL	http://localhost:8082
API version	v1
Resource	products
Base endpoint	/api/v1/products
Data format	application/json
Port	8082

API Design Principles

• Standard HTTP methods and status codes.

• Resource-oriented URLs.

• Explicit API version /api/v1.

• JSON request and response bodies.

• Consistent validation and error responses.

2. Product Resource

The Product resource represents an item managed by the Product Service.

Field	Description
id	Unique product identifier.
name	Product name.
description	Detailed product description.
price	Product price.
category	Product category.
stock	Available stock quantity.
status	Product status.
createdAt	Creation date/time.
updatedAt	Last update date/time.

3. API Endpoints

The Product Service exposes these six required REST endpoints.

Method	Endpoint	Purpose	Success
POST	/api/v1/products	Create product	201 Created
GET	/api/v1/products	Get all products	200 OK
GET	/api/v1/products/{id}	Get product by ID	200 OK
PUT	/api/v1/products/{id}	Complete update	200 OK
PATCH	/api/v1/products/{id}	Partial update	200 OK
DELETE	/api/v1/products/{id}	Delete product	204 No Content

3.1 POST /api/v1/products — Create Product

Full URL: **http://localhost:8082/api/v1/products**

Request body:

```
{ "name": "Gaming Laptop", "description": "High-performance gaming laptop", "price": 90000,
"category": "Electronics", "stock": 25 }
```

Success: 201 Created

```
{ "id": 1, "name": "Gaming Laptop", "description": "High-performance gaming laptop", "price":
90000, "category": "Electronics", "stock": 25 }
```

3.2 GET /api/v1/products — Get All Products

Full URL: **http://localhost:8082/api/v1/products**

Success: 200 OK

```
[ { "id": 1, "name": "Gaming Laptop", "description": "High-performance gaming laptop", "price":
90000, "category": "Electronics", "stock": 25 } ]
```

3.3 GET /api/v1/products/{id} — Get Product by ID

Example: **GET http://localhost:8082/api/v1/products/1**

Success: 200 OK

```
{ "id": 1, "name": "Gaming Laptop", "description": "High-performance gaming laptop", "price":
90000, "category": "Electronics", "stock": 25 }
```

Not found: 404 Not Found

```
{ "status": 404, "message": "Product not found with id: 999", "timestamp":
"2026-09-08T13:14:00" }
```

3.4 PUT /api/v1/products/{id} — Complete Update

Example: **PUT http://localhost:8082/api/v1/products/1**

```
{ "name": "Gaming Laptop Pro", "description": "Updated high-performance gaming laptop",
"price": 95000, "category": "Electronics", "stock": 30 }
```

Success: 200 OK. If the ID does not exist: 404 Not Found.

3.5 PATCH /api/v1/products/{id} — Partial Update

Example: **PATCH http://localhost:8082/api/v1/products/1**

```
{ "price": 92000, "stock": 28 }
```

Success: 200 OK. If the ID does not exist: 404 Not Found.

3.6 DELETE /api/v1/products/{id} — Delete Product

Example: **DELETE http://localhost:8082/api/v1/products/1**

Success: 204 No Content. The response has no body. If the ID does not exist: 404 Not Found.

4. Validation

Create and update requests use validation. Invalid data is rejected with HTTP 400 Bad Request.

Validation Rules

• Product name is required and must be between 2 and 100 characters.

• Product category is required and must be between 2 and 50 characters.

• Product price must be greater than or equal to 0.

• Product stock must be greater than or equal to 0.

Example invalid request:

```
{ "name": "", "price": -100, "stock": -5, "category": "" }
```

Response: 400 Bad Request

```
{ "status": 400, "message": "name: Product name must be between 2 and 100 characters, category:
Product category is required, price: Product price must be greater than or equal to 0, stock:
Product stock must be greater than or equal to 0", "timestamp": "2026-09-08T13:14:57" }
```

The global exception handler collects validation failures and returns a consistent error response.

5. Error Handling

A global exception handler provides consistent API errors instead of exposing raw framework exceptions.

Standard Error Response

```
{ "status": 400, "message": "Invalid product data", "timestamp": "2026-09-08T10:00:00" }
```

Handled error categories

• Validation exception — 400 Bad Request.

• Product not found — 404 Not Found.

• Duplicate product — 409 Conflict.

• Generic/unexpected exception — 500 Internal Server Error.

6. HTTP Status Codes

Status	Meaning	Typical Use
200 OK	Request successful	GET, PUT, PATCH
201 Created	Resource created	POST
204 No Content	Successful request with no body	DELETE
400 Bad Request	Invalid request / validation failure	POST, PUT, PATCH
404 Not Found	Product does not exist	ID-based operations
409 Conflict	Duplicate/conflicting product	Duplicate product
500 Internal Server Error	Unexpected server failure	Generic exception

7. Postman Testing Scenarios

Use Postman to verify the complete Product Service API.

#	Method	Scenario	Expected
1	POST	Valid product	201 Created
2	POST	Invalid name/price/stock/category	400 Bad Request
3	GET	Get all products	200 OK
4	GET	Existing product ID	200 OK
5	GET	Non-existing ID such as 999	404 Not Found
6	PUT	Valid complete update	200 OK
7	PUT	Non-existing ID	404 Not Found
8	PATCH	Valid partial update	200 OK
9	PATCH	Non-existing ID	404 Not Found
10	DELETE	Existing product	204 No Content
11	DELETE	Non-existing ID	404 Not Found
12	POST	Duplicate product scenario	409 Conflict

Recommended Test Order

• Create a valid product with POST.

• Use GET all products to confirm creation.

• Use GET by ID to confirm the resource.

• Test PUT and PATCH updates.

• Send invalid data and confirm 400.

• Request ID 999 (or another non-existing ID) and confirm 404.

• Delete the product and confirm 204.

• Verify the deleted product is no longer available.

8. API Summary

Method	Path	Purpose	Success
POST	/api/v1/products	Create	201
GET	/api/v1/products	Read all	200
GET	/api/v1/products/{id}	Read one	200
PUT	/api/v1/products/{id}	Complete update	200
PATCH	/api/v1/products/{id}	Partial update	200
DELETE	/api/v1/products/{id}	Delete	204

Base URL

```
http://localhost:8082/api/v1/products
```

9. API Versioning

The API uses URL-based versioning with **/api/v1**. A future breaking contract can be introduced as **/api/v2** while allowing v1 clients to continue using the existing contract.

• Clients can clearly identify the API contract.

• Future breaking changes can use a new version.

• Existing v1 clients can continue using v1.

• The version is easy to identify in Postman and documentation.

10. Step 9 Completion Checklist

Requirement	Included
Service overview, base URL, port 8082	Yes
Product resource fields	Yes
All six endpoints	Yes
Methods and full URLs	Yes
Request and response examples	Yes
Success responses	Yes
404 Product Not Found example	Yes
HTTP status codes 200/201/204/400/404/409/500	Yes
Validation rules and example	Yes
Global exception/error handling	Yes
Postman test scenarios	Yes
API summary	Yes
API versioning rationale	Yes

Step 9 is complete. This PDF is the formal API documentation for the Product Service and can be kept with the project documentation.